RAPPORT SAE 1.01

"Candy Crush"

Étudiants : BULUT Mélissa & DAMEZ Jean G4a

Numéro d'équipe : 5

Sommaire :

1. Introduction
2. Développement
   - 2.1 Exercices du Prog#10
3. Modes de jeu
   - 3.1 Mode Classique
   - 3.2 Mode PvP
   - 3.3 Mode Histoire
4. Problèmes rencontrés
5. Robustesse
6. Règles du jeu
7. Doxygen
   - 7.1 Annotation du code source
   - 7.2 Doxywizard
8. Conclusion
9. Bibliographie

1. Introduction :

Dans le cadre de la SAE 1.01, notre projet est de développer une version dans la console du célèbre jeu "Candy Crush". Ce projet s'appuie sur les bases acquises lors du Prog#10 (Exercices 1, 2 et 3).

L'objectif était de manipuler des matrices de vecteurs pour la grille, de gérer l'aléatoire pour la rejouabilité et de proposer une interface utilisateur ergonomique et colorée en ligne de commande ainsi que différents modes de jeux et une documentation de tout le code.

2. Développement

2.1 Exercices du Prog#10

Nous avons commencé par implémenter les bases demandées par notre professeur (référencées sur le site Ens.Casali.me) :

- Gestion de la couleur : Utilisation des séquences d'échappement VT100. Nous avons utilisé les codes ANSI pour les couleurs de police (30-36) et de fond (40-46) en nous appuyant sur les ressources de Cplusplus.com. Nous avons choisi du Cyan (46) pour le menu et du vert pour les règles (42).
- Structure de données : Nous avons utilisé un `typedef vector<vector> mat`. Cette structure nous permet de manipuler la grille (`grid[i][j]`). De plus, pour gérer les coordonnées, nous avons défini une structure `struct maPosition` (contenant *`abs`* et `ord`).
- Logique de jeu : L'implémentation de *`makeAMove`* permet l'échange de positions, tandis que *`atLeastThreeInARow`* et *`atLeastThreeInAColumn`* parcourent la matrice pour détecter les alignements. Si une suite est trouvée, elles renvoient la position de départ et la longueur de la suite, ce qui déclenche ensuite les fonctions de suppression `removalInRow` et `removalInColumn`. Les cases tout en haut de la colonne, devenues vides après la chute, sont remplies avec la constante *`KImpossible`* (0).

3. Modes de jeu

Pour enrichir le jeu, nous avons implémenté trois modes distincts. Le calcul du score (pour tous les modes de jeu) n'est pas simplement le nombre de cases, mais le nombre de cases multiplié par la valeur du chiffre (ex: trois "6" rapportent 18 points).

3.1 Mode classique

C'est le mode standard pour tester le moteur de jeu. Le joueur a 10 coups pour maximiser son score.

3.2 Mode PvP

Nous avons implémenté un mode duel où deux joueurs en local peuvent s'affronter. Le mécanisme est le suivant : le jeu détecte le tour grâce au modulo (*coups % 2*).

3.3 Mode histoire

Nous avons créé un scénario "Apocalypse zombie" : la ville est dans le noir et le joueur doit rétablir le courant afin de repousser les zombies.

- Structure : 5 niveaux progressifs.
- Objectifs : Chaque niveau possède un *objectifScore* croissant (de 20 à 100 points).
- Contrainte : Le joueur dispose d'une énergie limitée (*KMaxTimes*). Si l'objectif n'est pas atteint à temps, c'est la fin de la partie, nous sommes dévorés.

4. Problèmes rencontrés

Nous avons rencontré plusieurs difficultés majeures :

1. L'aléatoire répétitif : Au début, la grille était identique à chaque lancement. Il était donc fonctionnel mais jouable une seule fois. Nous avons découvert qu'il fallait "initialiser" le générateur de nombres. Nous avons trouvé la solution sur Cplusplus.com - srand en utilisant *srand(time(NULL))*, cela initialise le générateur avec l'heure actuelle, garantissant une grille unique à chaque partie.

2. Le buffer du cin : Lors de la lecture des commandes ou des pauses entre les niveaux, le programme "sautait" des étapes sans attendre l'utilisateur. Ce problème de reste de caractères dans le flux d'entrée a été résolu grâce à la fonction *cin.ignore()* documentée ici : Cplusplus.com - ignore. Cette fonction nous permet de vider le buffer pour forcer une véritable pause.

3. Lecture du fichier texte : Nous devions afficher les règles à partir d'un fichier texte (regles.txt), mais nous ne savions pas comment ouvrir un fichier en C++ pour lire son contenu. En cherchant une méthode pour ouvrir le fichier en lectuture seule, nous avons appris à utiliser la bibliothèque *<fstream>* et la classe *ifstream*. Nous avons suivi le tutoriel de OpenClassrooms - fstream. Nous avons également mis en place une sécurité : le programme tente d'ouvrir le fichier avec *file.is_open()*. S'il réussit, il l'affiche. S'il échoue (fichier introuvable ou mauvais chemin), il affiche une version simplifiée des règles via des *cout*. Cela permet au programme d'empêcher le crash.

4. Choix du design : Une difficulté est apparue lors de l'ajout des couleurs de fond. Pour les modes Classique et PvP, nous avons décidé de ne pas utiliser de couleur de fond. En effet, les chiffres étant déjà colorés, l'ajout d'un fond rendait la grille difficile à voir.

5. Robustesse

Nous avons veillé à éviter les erreurs simples qui peuvent faire planter le code :

- Vérification systématique des indices de matrice pour éviter les débordements (*newR < taille, newC > 0*).
- Nettoyage des entrées utilisateurs (*cin.clear()*) pour éviter les boucles infinies en cas de saisie de texte à la place d'un chiffre.

6. Règles du jeu

Les règles du jeu ainsi que le but sont présentes dans le fichier regles.txt joint. En résumé :

- Déplacement : Z (Haut), S (Bas), A (Gauche), E (Droite).
- Quitter : Tapez 'M' au moment de saisir la ligne.

7. Doxygen

7.1 Annotation du code source

Au début, nous pensions que Doxygen analysait le code seul. Grâce aux tutoriels (notamment sur Developpez.com), nous avons compris qu'il fallait utiliser une syntaxe spécifique directement dans nos fichiers *.cpp*. Nous avons donc repris toutes nos fonctions pour y ajouter des blocs de commentaires au format "Javadoc" (/** ... */). Nous avons appris à utiliser les balises standardisées :

@brief pour décrire rapidement le but de la fonction.

@param pour expliquer chaque paramètre (ex: grid, pos).

@return pour préciser ce que la fonction renvoie (vrai/faux, un score, etc.).

7.2 Doxywizard

L'installation de l'outil et de son extension Graphviz (nécessaire pour les schémas) a été complexe au départ. Nous nous sommes aidés d'une vidéo explicative sur YouTube pour configurer correctement les chemins d'accès (DOT_PATH). Pour obtenir le rendu demandé par le sujet, nous avons réglé Doxywizard ainsi :

- Graphes : Avctivation des options HAVE_DOT, CALL_GRAPH et CALLER_GRAPH
- Profondeur : La profondeur maximale des graphes a été fixée à 4.

8. Conclusion

Ce projet nous a permis de comprendre l'importance d'une structure de code propre et documentée. Travailler en binôme nous a forcés à être rigoureux sur les noms de fonctions et les commentaires. Nous avons maintenant un jeu fonctionnel, documenté et avec plusieurs modes de jeux.

9. Bibliographie :

Support de cours et sujet :

https://ens.casali.me/category/r1-01-init-dev/r1-01-programmation/r101-prog10-2021/

https://ens.casali.me/category/r1-01_sae/r1-01-sae-1-01-r1-01_sae/

Ressources Techniques :

Srand : https://cplusplus.com/reference/cstdlib/srand/

Cin.ignore : https://cplusplus.com/reference/istream/istream/ignore/

Fstream : https://openclassrooms.com/fr/courses/1894236-apprenez-a-programmer-en-c/7534171-lisez-et-modifiez-des-fichiers

Code couleurs ANSI : https://cplusplus.com/forum/unices/36461/

Documentation :

Doxygen : https://www.doxygen.nl

Graphviz : https://graphviz.org